

Program Specification: UADC Supporting Tools

1. Purpose

This document specifies every supporting program under **tools/**. It records the role, inputs, outputs, command-line interface, processing sequence, major functions and data structures, validation behavior, and operational constraints of each tool at a consistent level of detail.

The tools support four activities:

- EN 16931 LHM construction and maintenance;
- syntax-binding preparation and tutorial conversion;
- taxonomy, metadata, and round-trip artifact generation;
- inspection of generated flat files.

Operational Phase 1 and Phase 2 converters remain under **src/**. Programs under **tools/** either prepare their definitions and test artifacts or provide smaller tutorial implementations.

2. Scope

The specification covers these 15 programs:

No.	Program	Classification
1	audit_en16931_coverage.py	LHM audit
2	build_lhm_from_source.py	LHM generation
3	build_roundtrip_test_artifacts.py	Test-artifact generation
4	build_syntax_binding.py	Binding generation
5	check_lhm_class_element.py	LHM validation
6	extend_en16931_lhm_coverage.py	LHM maintenance
7	normalize_lhm_class_element.py	LHM normalization
8	normalize_lhm_semantic_paths.py	LHM normalization
9	order_lhm_by_en16931_table2.py	LHM ordering
10	psv_viewer.html	Browser inspection
11	tutorial/semantic_binding_sample.py	Tutorial semantic conversion
12	tutorial/syntax_binding_sample.py	Tutorial syntax conversion
13	taxonomy/xBRLGL_TaxonomyGenerator.py	Taxonomy generation
14	update_lhm_definitions_from_pdf.py	LHM definition update

No.	Program	Classification
15	update_lhm_syntax_sequence_from_ubl_xsd.py	LHM syntax-order update

3. Shared Conventions

3.1 Execution Directory

Commands in this document are run from the UADC PoC repository root.

Windows PowerShell:

```
$python = 'python'
& $python .\tools\PROGRAM.py [arguments]
```

macOS or Linux:

```
PYTHON=python3
$PYTHON ./tools/PROGRAM.py [arguments]
```

3.2 CSV Encoding

Tools with an encoding option default to **utf-8-sig**. This preserves a UTF-8 byte-order mark for compatibility with spreadsheet software. Tools without an encoding option also read and write LHM CSV files as **utf-8-sig**.

3.3 Exit Status

The normal convention is:

- exit status **0**: processing or validation succeeded;
- exit status **1**: a handled validation failure or conversion error occurred;
- uncaught exception: an input file, schema, required column, or dependency is missing or invalid.

3.4 In-place Modification

The following programs modify their input LHM CSV directly:

- **extend_en16931_lhm_coverage.py**;
- **normalize_lhm_class_element.py**;
- **order_lhm_by_en16931_table2.py**;
- **update_lhm_definitions_from_pdf.py**.

normalize_lhm_semantic_paths.py also modifies the input in place when **--output** is omitted. Review or copy the source before running these tools.

4. Tool Specifications

4.1 audit_en16931_coverage.py

Role

Audits whether every EN 16931 **BG-n** and **BT-n** identifier found in the standard PDF is represented in the LHM CSV.

Inputs and Outputs

- Input 1: EN 16931-1 PDF.
- Input 2: LHM CSV containing an **id** column.
- Output: a JSON report written to standard output.
- Side effects: none.

Command

```
& $python .\tools\audit_en16931_coverage.py STANDARD.pdf LHM.csv
```

Processing

1. **extract_pdf_ids** reads every PDF page with **pypdf.PdfReader**.
2. The regular expression **BG-n** or **BT-n** collects the first page on which each identifier appears.
3. **read_lhm_ids** collects matching identifiers from the LHM **id** column.
4. The program calculates missing and extra identifier sets.
5. It prints counts, missing identifiers with PDF page numbers, and extra LHM identifiers as formatted JSON.

Functions

Function	Responsibility
sort_key	Sorts identifiers by BG or BT prefix and numeric suffix.
extract_pdf_ids	Extracts identifier-to-first-page mappings from the PDF.
read_lhm_ids	Reads valid BG and BT identifiers from the LHM.
main	Parses inputs, compares sets, prints JSON, and returns status.

Exit and Dependencies

Returns **1** when at least one PDF identifier is missing from the LHM; otherwise returns **0**. Requires **pypdf**. Extra LHM identifiers are reported but do not by themselves cause failure.

4.2 build_lhm_from_source.py

Role

Maintains a separation between an editable EN 16931 source table and the normalized LHM CSV consumed by conversion and taxonomy scripts.

Inputs and Outputs

The **init-source** command reads a generated LHM and writes an editable source CSV. The **build** command reads that editable source and writes a generated LHM with normalized types, multiplicities, hierarchy levels, semantic paths, class terms, and unique element names.

Commands

```
& $python .\tools\build_lhm_from_source.py init-source LHM.csv SOURCE.csv  
& $python .\tools\build_lhm_from_source.py build SOURCE.csv LHM.csv
```

Processing

For **init-source**, **source_from_lhm** copies reviewable business fields and creates override columns.
For **build**:

1. Source rows are sorted by **sequence**.
2. **type_from_id** derives class or attribute type when type is absent.
3. **normalize_multiplicity** converts source cardinality to the supported LHM representation.
4. **semantic_path** uses an explicit override or assembles lower-camel-case segments following **path** identifiers.
5. **nearest_bg_id** selects the semantic class for each fact.
6. **lhm_effective_levels** assigns levels only to the invoice root, repeated classes, and facts owned by the nearest effective repeated ancestor.
7. Explicit class-term and element overrides are retained; missing elements are generated by **unique_element_names**.
8. Duplicate elements and unsupported multiplicities are rejected before the LHM is written.

Functions

Function	Responsibility
normalize_multiplicity	Normalizes cardinality notation.
read_rows, write_rows	Read and write UTF-8 BOM CSV records.
type_from_id	Maps BG to class and BT to attribute rows.
lower_camel_case_concatenated	Creates semantic path segments.
source_from_lhm	Creates the editable source form.
path_ids	Returns the hierarchy identifiers for one row.

Function	Responsibility
semantic_path	Builds or applies the semantic path.
nearest_bg_id	Finds the owning business group.
lhm_effective_levels	Calculates effective hierarchical CSV levels.
build_lhm	Orchestrates generated LHM construction.
validate_unique_elements	Rejects duplicate concept names.
validate_multiplicities	Rejects unsupported multiplicities.
main	Dispatches init-source or build .

Validation

After generation, run the LHM semantic-path, hierarchical-layout, and taxonomy generator regression tests. The generated LHM is authoritative; manual changes should normally be made in the editable source CSV or override columns.

4.3 build_roundtrip_test_artifacts.py

Role

Rebuilds the complete Phase 1 round-trip artifact sets used by regression tests.

Inputs and Outputs

Inputs are fixed by **DATASETS**:

- the minimal OpenPeppol UBL Invoice sample;
- all UBL Invoice XML files under the BIS Billing 3 sample directory.

For each dataset it recreates:

```
tests/roundtrip/DATASET/source_xml/  
tests/roundtrip/DATASET/structured_csv/  
tests/roundtrip/DATASET/metadata_json/  
tests/roundtrip/DATASET/roundtrip_xml/
```

Command

```
& $python .\tools\build_roundtrip_test_artifacts.py
```

The program has no command-line options. Supplying **--help** still runs the builder because the script does not use an argument parser.

Processing

1. **ensure_taxonomy** verifies the OIM entry schema and EN 16931 module schema; if absent, it invokes the taxonomy generator regression script.
2. **is_invoice_xml** excludes XML documents whose root element is not Invoice.
3. **clean_directory** removes existing files from each target artifact folder.
4. **build_dataset** copies source XML, generates Structured CSV and metadata JSON, then reverse-converts the CSV into UBL XML.
5. **run_converter** invokes **src/syntax_binding.py** through the current Python executable and fails immediately when the child process fails.

Functions

Function	Responsibility
ensure_taxonomy	Ensures required generated taxonomy schemas exist.
is_invoice_xml	Checks the XML root local name.
clean_directory	Recreates and empties one artifact directory.
run_converter	Runs forward or reverse syntax binding.
build_dataset	Builds all four artifacts for one sample collection.
main	Builds both configured datasets and reports the total.

Constraints

This is a regenerating tool. Existing files directly inside the four target artifact folders are deleted before rebuilding. Run the round-trip artifact, UBL schema, and xBRL-CSV metadata validation scripts after generation.

4.4 build_syntax_binding.py

Role

Converts a simple table of **semantic_path** and **xpath** pairs into either a compact binding CSV or an LHM/HMD-style syntax binding.

Inputs and Outputs

- Input: CSV containing both **semantic_path** and **xpath**.
- Output: binding CSV selected by **--output**.
- Default encoding: **utf-8-sig**.

Command

```
& $python .\tools\build_syntax_binding.py BINDINGS.csv `
  --output SYNTAX_BINDING.csv [--simple] [--encoding utf-8-sig]
```

Without **--simple**, the output contains generated class rows and attribute rows in the full LHM header. With **--simple**, the output contains only **column**, **semantic_path**, and **xpath**.

Processing

1. **read_source_rows** validates required columns and rejects incomplete pairs.
2. **split_semantic_path** splits on dots outside predicate brackets.
3. **clean_token** and **semantic_path_to_column** create safe stable columns.
4. **unique_name** adds numeric suffixes to collisions.
5. **build_simple_rows** emits one row per input pair, or **build_lhm_rows** creates missing ancestor class rows followed by attribute rows.
6. **build_syntax_bindings** creates the destination directory and writes rows.

Functions

Function	Responsibility
semantic_path_to_column	Derives a target column from a semantic path.
split_semantic_path	Preserves predicates while splitting path segments.
segment_label, segment_element	Derive labels and element tokens.
clean_token, unique_name	Sanitize and deduplicate identifiers.
read_source_rows	Validates source pairs.
build_simple_rows	Produces the compact format.
build_lhm_rows	Produces generated class and attribute rows.
build_syntax_bindings	Writes the selected output format.
main	Parses arguments and converts handled exceptions to status 1 .

Constraints

Generated class multiplicity is **1..***, generated fact multiplicity is **0..1**, and generated fact datatype is Text. Review these defaults before using the result as a production binding.

4.5 check_lhm_class_element.py

Role

Validates that LHM **class_term** and **element** values match the normalization rules without modifying the CSV.

Command and Input

```
& $python .\tools\check_lhm_class_element.py LHM.csv
```

Processing

1. Reads the LHM and maps identifiers to names.
2. Reuses **nearest_parent_bg**, **singularize**, and **unique_element_names** from **normalize_lhm_class_element.py**.
3. Compares expected and stored values for BG and BT rows.
4. Prints up to 50 mismatches in each category and a total count.

Functions

Function	Responsibility
read_rows	Reads normalized LHM dictionaries.
main	Calculates expected values, reports differences, and returns status.

Returns **1** if either mismatch list is non-empty; otherwise prints the checked row count and returns **0**.

4.6 extend_en16931_lhm_coverage.py

Role

Applies the program's curated EN 16931 coverage additions and corrections to an existing LHM CSV.

Command and Side Effect

```
& $python .\tools\extend_en16931_lhm_coverage.py LHM.csv
```

The input file is rewritten in place.

Processing

1. Static **UPDATES** amend known rows by identifier.
2. Static **ROWS** define missing BG and BT records.
3. Existing identifiers are not duplicated.
4. **write_rows** rennumbers **sequence** and recalculates **level** from **path**.

Functions and Data

Item	Responsibility
class_row, attr_row, row	Build normalized curated records.
UPDATES	Field corrections for existing identifiers.
ROWS	Curated records to add when absent.
read_rows, write_rows	Read and rewrite the full LHM.
main	Applies updates and reports the number of additions.

Constraints

The program is data-driven but not a general PDF parser. After execution, run the ordering, semantic-path normalization, class/element normalization, and LHM checks so derived fields remain consistent.

4.7 normalize_lhm_class_element.py

Role

Normalizes LHM **class_term** and **element** values in place.

Command

```
& $python .\tools\normalize_lhm_class_element.py LHM.csv
```

Rules

- A BG row belongs to its own group; a BT row belongs to its nearest BG ancestor.
- **class_term** is the singularized owning BG business term.
- **element** starts with an uppercase character and is derived from the shortest unique suffix of **semantic_path**.
- Duplicate names are expanded by adding more semantic path segments.

Functions

Function	Responsibility
singularize	Applies the English singularization rules.
semantic_segments	Splits a semantic path into name segments.
upper_camel	Converts one segment to UpperCamelCase.
suffix_element_name	Builds an element from a path suffix.

Function	Responsibility
unique_element_names	Finds the shortest unique name for every row.
nearest_parent_bg	Finds the owning BG identifier.
read_rows, write_rows	Read and rewrite the input CSV.
main	Updates changed cells and reports the value count.

Run **check_lhm_class_element.py** immediately after normalization.

4.8 normalize_lhm_semantic_paths.py

Role

Rebuilds every LHM **semantic_path** from row names and the identifier hierarchy.

Command

```
& $python .\tools\normalize_lhm_semantic_paths.py INPUT.csv `
  [--output OUTPUT.csv] [--encoding utf-8-sig]
```

When **--output** is omitted, **INPUT.csv** is replaced.

Processing

1. **lower_camel_case_concatenated** converts each business term to one path segment, using **unnamed** for empty terms and prefixing numeric starts.
2. **normalize_rows** maps row identifiers to segments.
3. For each row, segments are selected in **path** order.
4. The resulting value is written as **\$.segment.child**.
5. **normalize_file** retains the original CSV column order.

Functions

Function	Responsibility
lower_camel_case_concatenated	Creates lower-camel-case segments.
normalize_rows	Replaces semantic paths in memory.
normalize_file	Reads, normalizes, and writes a complete CSV.
main	Selects output location and handles errors.

Explicit semantic-path overrides are not consulted by this standalone tool. Use **build_lhm_from_source.py** when editable source overrides must be retained.

4.9 order_lhm_by_en16931_table2.py

Role

Orders all LHM rows according to the embedded EN 16931-1 Table 2 identifier sequence and applies known hierarchy corrections.

Command and Side Effect

```
& $python .\tools\order_lhm_by_en16931_table2.py LHM.csv
```

The input file is rewritten in place.

Processing

1. **FIXES** updates known row fields before sorting.
2. **TABLE2_ORDER** supplies the complete identifier order beginning with the invoice root.
3. Any identifier absent from the order list raises an error; unknown rows are never silently placed at the end.
4. Rows are sorted, **sequence** is renumbered, and **level** is recalculated from **path**.

Functions and Data

Item	Responsibility
TABLE2_ORDER	Defines normative program ordering.
FIXES	Applies curated hierarchy or field corrections.
read_rows, write_rows	Read and rewrite ordered records.
main	Validates completeness, sorts, and reports the row count.

4.10 psv_viewer.html

Role

Provides a standalone browser viewer for generated ADS PSV, CSV, tab-delimited, or text files. It does not upload data and requires no web server.

User Inputs and Outputs

- Input: local file selected or dropped into the page.
- Delimiter: pipe, comma, or tab.
- Filter: case-insensitive text applied to rendered rows.
- Output: an interactive HTML table in the browser only.

Processing

1. **loadFile** reads the selected file with **FileReader** and removes a BOM.
2. **normalizedDelimiter** maps the UI choice to the actual delimiter character.
3. **parseDelimited** parses quoted cells, escaped quotes, delimiters, and line endings without a server-side library.
4. **renderTable** treats the first row as headers, adds row numbers, and hides columns empty in every data row.
5. **applyFilter** toggles rows based on searchable lower-case row text.
6. **updateMeta** displays file name, row count, column count, hidden columns, and visible-row count.

Functions

Function	Responsibility
parseDelimited	Parses delimiter-separated text with quote handling.
normalizedDelimiter	Resolves pipe, comma, or tab.
renderMessage	Displays empty-file or warning messages.
renderTable	Builds the sticky-header HTML table.
updateMeta	Updates file and table statistics.
applyFilter	Filters rows without reparsing the file.
loadFile	Coordinates local file reading and rendering.

Constraints

The viewer is for inspection, not validation or editing. All processing occurs in browser memory. Closing or refreshing the page discards the loaded data.

4.11 tutorial/semantic_binding_sample.py

Role

Demonstrates a simple row-for-row Structured-CSV-to-flat-CSV semantic binding. It is not the operational Phase 2 converter. Production processing uses **src/semantic_binding.py**, which understands class multiplicity, repeated row scope, indexed semantic paths, target formats, and directory input.

Command

```
& $python .\tools\tutorial\semantic_binding_sample.py STRUCTURED.csv `
--binding BINDING.csv --output FLAT.csv [--encoding utf-8-sig]
```

Binding Compatibility

The reader accepts legacy alternatives for target name, source path, and fixed value columns. The final source segment after a slash or dot is used as the Structured CSV column name.

Processing

1. **read_bindings** selects rows having a target and either source or fixed value.
2. **tail** reduces the source path to a column name.
3. The input Structured CSV is read as independent rows.
4. For every input row, each target receives its source cell or fixed fallback.
5. One output row is written for every input row.

Functions

Function	Responsibility
first_present	Reads the first populated legacy field name.
tail	Extracts the final path segment.
read_bindings	Normalizes usable binding rows.
convert_structured_to_flat	Performs row-for-row projection.
main	Parses arguments, reports dimensions, and handles errors.

Constraints

This tutorial does not merge hierarchical rows, infer repeated ancestors, or enforce the single-child versus repeated-child Structured CSV rules.

4.12 tutorial/syntax_binding_sample.py

Role

Demonstrates a compact XML-to-CSV syntax-binding implementation. It is not the operational Phase 1 converter. Production conversion and reverse conversion use **src/syntax_binding.py**.

Command

```
& $python .\tools\tutorial\syntax_binding_sample.py INPUT.xml `
  --binding BINDING.csv --output OUTPUT.csv `
  [--row-xpath XPATH] [--encoding utf-8-sig]
```

Processing

1. **collect_namespaces** reads namespace declarations from the source XML.
2. **read_bindings** accepts several legacy binding column names.
3. **find_nodes** implements a limited child-step XPath evaluator.

4. Optional **--row-xpath** chooses repeated row contexts; without it the document root produces one row.
5. Each binding extracts an element text or terminal attribute; multiple text matches are pipe-joined.
6. A binding default is used when extraction returns an empty value.

Supported Predicates

The limited evaluator supports:

- child-path equality or inequality with a quoted literal;
- equality or inequality with an absolute document path;
- child-path comparison with **true()** or **false()**;
- terminal attributes and namespace-prefixed element steps.

Unsupported predicates are not rejected and currently behave as matches. This is an important reason not to use the sample converter as the runtime authority.

Functions

Function group	Responsibility
local_name, collect_namespaces, qualify_step	Namespace handling.
split_step_predicate, split_xpath, split_terminal_attribute	XPath parsing.
path_value, predicate_matches, child_matches	Predicate evaluation.
find_nodes, get_value	XML node and scalar extraction.
first_present, read_bindings	Legacy binding normalization.
write_structured_csv	Emits one row per selected context.
main	Parses arguments and handles conversion errors.

4.13 taxonomy/xBRLGL_TaxonomyGenerator.py

Role

Generates the local XBRL-CSV taxonomy, dimensional definition relationships, presentation relationships, labels, metadata template, and CSV skeleton from an LHM/HMD-style definition CSV.

Command

```
& $python .\tools\taxonomy\xBRLGL_TaxonomyGenerator.py LHM.csv `
--base_dir out\taxonomy `
--palette en16931 `
```

```
--root invoice `
--lang en `
--currency JPY `
--namespace http://www.xbrl.org/int/gl/plt/2026-07-05 `
[--encoding utf-8-sig] [--trace] [--debug]
```

Arguments

Argument	Meaning
inFile	Input LHM/HMD CSV.
--base_dir	Generated taxonomy root; default is the current directory.
--palette	Default module when a record has no module.
--root	Root semantic identifier used for metadata columns.
--lang	Local label language; default ja .
--currency	ISO 4217 unit for amount columns; default JPY .
--namespace	Taxonomy namespace; final ten characters define the version.
--encoding	CSV and generated text encoding.
--trace, --debug	Diagnostic output controls.

Processing

1. The constructor validates the input and creates the output root.
2. **load_csv_data** accepts the UADC LHM header, normalizes records, maps datatypes, derives presentation parents, and builds effective dimension ownership from **lhm_level** and multiplicity.
3. Only facts and effective dimension classes are retained for taxonomy output.
4. **process_records** derives repeated-class parent relationships and role data.
5. **generate_taxonomy_files** writes module schemas, OIM entry schema, presentation, labels, and dimensional definition relationships.
6. **ensure_gl_gen_schema** locates or versions the generic datatype schema.
7. **json_meta_file** writes an xBRL-CSV JSON template and CSV skeleton when a root is supplied.

Primary Generated Files

For the EN 16931 profile and version **2026-07-05**, the important files are:

```
out/taxonomy/en16931/en16931-2026-07-05.xsd
out/taxonomy/gen/gl-gen-2026-07-05.xsd
out/taxonomy/plt/en16931-oim-2026-07-05.xsd
out/taxonomy/plt/en16931-def-2026-07-05.xml
out/taxonomy/en16931/en16931-2026-07-05-presentation.xml
```

```
out/taxonomy/xbrl-gl.json
out/taxonomy/xbrl-gl_skeleton.csv
```

Class Methods

Method	Responsibility
init	Initializes paths, version, options, and ordered model maps.
debug_print, trace_print, error_print	Diagnostic and fatal reporting.
ensure_gl_gen_schema, gl_gen_schema_location	Provide the generic datatype schema.
concept_item_type	Maps LHM datatypes to XBRL item types.
LC3, titleCase, SC, escape_text	Name and text normalization.
getRecord, getParent, getChildren, getElementID	Model lookup.
domainMember, defineHypercube	Build dimensional relationships.
roleRecord, linkPresentation	Build role and presentation structures.
normalize_lhm_record	Adapts UADC and legacy LHM columns.
load_csv_data	Loads records and builds presentation/dimension maps.
process_records	Finalizes repeated-class and role relationships.
generate_taxonomy_files	Writes schemas and linkbases.
json_meta_file	Writes metadata and the optional CSV skeleton.

Validation and Constraints

Run **tests/test_xbrlgl_generator_uadc_lhm.py** and **tests/validate_taxonomy.py**, then validate metadata with Arelle. The OIM entry point is **en16931-oim-2026-07-05.xsd** and the dimensional definition linkbase is **en16931-def-2026-07-05.xml**. Tuple/content entry schemas are not part of this PoC profile.

4.14 update_lhm_definitions_from_pdf.py

Role

Updates LHM **definition** values from the Description column of EN 16931-1 Table 2.

Command


```
& $python .\tools\update_lhm_definitions_from_pdf.py STANDARD.pdf LHM.csv
[ --first-page 43 ] [ --last-page 75 ]
```

The LHM is rewritten in place. Page numbers are one-based.

Processing

- 1. **extract_descriptions** uses **pdfplumber** to inspect tables in the selected page range.
- 2. **clean_identifier** extracts BG and BT identifiers from the first cell.
- 3. **clean_description** normalizes whitespace and removes a Description label.
- 4. Description fragments continuing across table rows are concatenated.
- 5. **DESCRIPTION_OVERRIDES** supplies curated replacements for extraction gaps.
- 6. Matching LHM definitions are updated; missing extractions and unresolved empty definitions are reported separately.

Functions and Data

Item	Responsibility
clean_cell, clean_description, clean_identifier	Normalize PDF cells.
extract_descriptions	Extract identifier-to-description mappings.
DESCRIPTION_OVERRIDES	Fill known PDF extraction gaps.
read_rows, write_rows	Read and rewrite the LHM.
main	Applies extracted definitions and reports unresolved rows.

Exit and Dependency

Returns **1** only when at least one BG or BT row still has an empty definition. Requires **pdfplumber**. A description that could not be extracted but already has a non-empty LHM value is reported without causing failure.

4.15 update_lhm_syntax_sequence_from_ubl_xsd.py

Role

Populates LHM **syntax_sequence** from the actual child order in extracted OASIS UBL 2.1 XML Schema files.

Command

```
& $python .\tools\update_lhm_syntax_sequence_from_ubl_xsd.py INPUT.csv `
--output OUTPUT.csv `
--schema-root UBL-2.1\xsd `
[--encoding utf-8-sig]
```

The program does not download UBL schemas.

Processing

1. **SchemaIndex.load_directory** recursively loads every XSD.
2. Global elements, named complex types, inline sequences, and extension sequences are indexed by namespace/local-name pairs.
3. XPath predicates and terminal attributes are removed from structural steps.
4. **syntax_sequence_for_xpath** follows the UBL element declarations and records each one-based child position as four digits, beginning with **0000** for the document root.
5. A terminal attribute is appended as **@name**.
6. Rows are sorted by resolved syntax sequence; unresolved rows retain source order after resolved rows.
7. The column is inserted after **sequence** when absent.

Data Structures and Functions

Item	Responsibility
ElementDecl	Stores an element QName and optional type QName.
SchemaIndex	Indexes XSD elements, types, and child sequences.
clean_xpath_step, split_xpath_steps	Parse structural XPath steps.
split_terminal_attribute, xpath_steps	Separate terminal attributes.
step_qname	Resolves UBL prefixes to namespaces.
syntax_sequence_for_xpath	Traverses the schema order for one XPath.
sort_key	Sorts resolved before unresolved records.
update_lhm	Loads schemas, updates all rows, and writes the output.
main	Parses arguments and reports resolved/total counts.

Validation and Constraints

An unresolved XPath receives a blank sequence but does not cause status **1**. Review the resolved count and run **tests/test_ubl_schema_child_order.py**. Store downloaded schema packages under a

cache or generated-output directory unless their licensing and repository role explicitly require commitment.

5. Recommended Maintenance Workflow

For a source-model change:

1. Update the editable source CSV or curated coverage data.
2. Build or extend the LHM.
3. Normalize semantic paths, class terms, and elements as appropriate.
4. Update syntax order from the UBL schema.
5. Order rows by EN 16931 Table 2.
6. Run the class/element and coverage audits.
7. Generate and validate the taxonomy.
8. Rebuild and validate round-trip artifacts.

For a binding tutorial or prototype:

1. Generate a syntax binding with **build_syntax_binding.py**.
2. Inspect its generated defaults and multiplicities.
3. Exercise it with **tools/tutorial/syntax_binding_sample.py**.
4. Move reviewed binding definitions to **specs/bindings/**.
5. Use the operational **src/** converter and its regression tests for PoC deliverables.

6. Validation Matrix

Changed area	Minimum checks
LHM identifiers or hierarchy	test_lhm_semantic_paths.py , test_lhm_hierarchical_csv_layout.py
Class terms or elements	check_lhm_class_element.py
UBL syntax order	test_ubl_schema_child_order.py
Taxonomy generation	test_xbrlgl_generator_uadc_lhm.py , validate_taxonomy.py
Structured CSV or metadata	test_syntax_binding.py , test_xbrl_csv_metadata_arelle.py
Reverse conversion	test_syntax_binding_reverse.py , test_roundtrip_xml_ubl_schema.py
Phase 2 bindings	Target-specific ADS tests and semantic binding tests

7. Dependencies

Most tools use only the Python standard library. Additional dependencies are:

Dependency	Used by
pypdf	audit_en16931_coverage.py
pdfplumber	update_lhm_definitions_from_pdf.py
Browser JavaScript APIs	psv_viewer.html
Arelle, external validation	Generated taxonomy and metadata validation tests

The taxonomy generator uses local schema resources and does not download them. The round-trip builder invokes repository scripts through the same Python executable used to start the builder.

8. Maintenance Rules

- Keep the tool inventory in this document synchronized with **tools/**.
- Update the relevant subsection whenever arguments, functions, generated files, dependencies, or exit behavior change.
- Do not describe tutorial converters as operational runtime converters.
- Record in-place modification and directory-cleaning behavior explicitly.
- Regenerate this document's PDF with VSCode Markdown PDF after Markdown edits.
- Use the configured margins: top and bottom **20mm**, left and right **18mm**.